

Keywords: code generation • reinforcement learning • runtime efficiency • execution feedback

RLPF: Reinforcement Learning from Performance Feedback for Code Generation

A Staged Reward Trains Code Models to Prefer Faster Correct Programs—Raising Correct-and-Efficient Rate from 8.1% to 38.6%

By Huihao Jing, Haozhe Cui, Wenbin Hu, Shaojin Chen, Haochen Shi, Changxuan Fan, Yuxuan Liu, Hanyu Yang, Sirui Zhang, Ziyi Chen, Haoran Li, Yangqiu Song (Hong Kong Univ. of Science and Technology)

arXiv:2607.27271 • July 29, 2026

Code models trained with execution feedback reward correctness—not efficiency. Two programs can pass the same test suite while differing by orders of magnitude in runtime. **RLPF** (Reinforcement Learning from Performance Feedback) introduces a staged reward: correctness is gated first; passing programs then receive a speedup signal relative to a reference solution. On a competitive programming benchmark, RLPF raises a base model from 8.1% to 38.6% CGRE (Correct-and-Graded Relative Efficiency), closing two-thirds of the gap to a frontier model.

AT A GLANCE

Problem: Correctness-only RL for code leaves no signal for runtime efficiency; programs that pass tests may be asymptotically slow.

Why it matters: Systems code, competitive programming, and latency-sensitive applications require efficient implementations as a primary objective.

Method: Two-stage reward: correctness gate first; relative speedup over a reference solution for programs that pass.

Result: Correct-and-runable rate 11.1%→54.6%; CGRE 8.1%→38.6%.

Evidence: Staged reward outperforms correctness-only RL by 7.3 CGRE points; validated across array, sorting, graph, and DP tasks.

The Big Picture

The code generation field has converged on a compelling training paradigm: train a language model on execution feedback. Sample programs from the model, execute them against a test suite, and use the pass/fail signal as a reward for reinforcement learning. This has driven dramatic improvements on correctness-centric benchmarks

such as HumanEval, MBPP, and LiveCodeBench.

But correctness is not the only property that matters. In systems programming, competitive programming, and any latency-sensitive application, a correct but slow program is often not good enough. Runtime efficiency should be a first-class training objective. RLPF is the first study that makes efficiency a primary RL training signal for code generation, rather than an evaluation-time metric.

Why Existing Approaches Fall Short

Correctness-only RL has a critical gap: the model learns to produce any program that passes the test suite, regardless of runtime. For most benchmarks, the test suite uses small inputs where even $O(n^3)$ solutions pass within the time limit. The model thus learns to write correct but asymptotically slow code.

Three properties of runtime make it a difficult RL reward:

- 1. Fragility.** Runtime is only meaningful after a program compiles and passes all correctness checks. If 80% of sampled programs fail to compile, a pure efficiency reward provides near-zero gradient.
- 2. Non-comparability across tasks.** An absolute runtime of 100 ms may be excellent for one problem and catastrophic for another. A relative metric is required.
- 3. Mode collapse risk.** Optimizing for efficiency without a correctness constraint causes the model to produce fast-but-wrong programs (e.g., always returning a hard-coded answer).

RLPF addresses all three with a staged reward design.

Core Mechanism

RLPF uses PPO with a two-stage reward function:

Stage 1 — Correctness gate. A sampled program $\hat{c}$ is executed against the full test suite. If it fails to compile, fails any test, or times out, it receives zero reward and the performance stage is skipped.

Stage 2 — Performance reward. If $\hat{c}$ passes all tests, its runtime $t(\hat{c})$ is measured on a distribution of inputs. The reward is the relative speedup over a reference correct solution:

$$r_{\text{perf}}(\hat{c}) = \frac{t_{\text{ref}} - t(\hat{c})}{t_{\text{ref}}}$$

This is bounded in $(-\infty, 1]$, with 0 meaning same speed as the reference and 1 meaning infinite speedup.

Technical Formulation

The combined reward is:

$$r(\hat{c}) = \alpha \cdot r_{\text{correct}} + (1 - \alpha) \cdot r_{\text{perf}}(\hat{c}) \cdot \mathbf{1}[r_{\text{correct}} = 1]$$

where α is annealed from 1.0 to 0.3 over training via a cosine schedule.

The PPO objective maximizes:

$$\mathcal{J}(\theta) = \mathbb{E}[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

where $\rho_t = \pi_\theta(\hat{c}|p)/\pi_{\text{old}}(\hat{c}|p)$ and A_t is the advantage from $r(\hat{c})$.

The evaluation metric CGRE is:

$$\text{CGRE} = \frac{1}{|P|} \sum_{p \in P} \mathbf{1}[\hat{c}_p \text{ passes}] \cdot \max\left(0, 1 - \frac{t(\hat{c}_p)}{t_{\text{ref}}(p)}\right)$$

This rewards both correctness and efficiency: failing programs contribute 0, and faster programs contribute proportionally more.

Learning or Inference Procedure

Training uses a dataset of competitive programming problems covering arrays, sorting, graph algorithms, and dynamic programming. Reference solutions are taken from accepted competitive submissions.

PPO is run for 3,000 steps with batch size 32, sampling 8 rollouts per problem per step. Each rollout is compiled and executed; the staged reward is computed online. The α annealing follows a cosine curve, ensuring the correctness stage dominates early training when the model produces mostly incorrect programs.

The KL divergence penalty $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ relative to the initial model prevents mode collapse onto a few fast-but-narrow templates.

What the Guarantee Says

The α annealing provides a correct-first guarantee: during the first phase of training (α near 1), the policy is incentivized exclusively to produce correct programs. By the time α decreases and the performance reward becomes significant, the correctness rate is already high enough that most sampled programs enter the performance stage, providing dense efficiency gradients.

Experimental Findings

Method	Correct-and-Run	CGRE
Base model	11.1%	8.1%
Correctness-only RL	47.3%	18.4%
RLPF (staged)	54.6%	38.6%
GPT-5.4 (zero-shot)	58.2%	41.1%

RLPF raises CGRE by 30.5 points over the base model and by 20.2 points over correctness-only RL. The gap to a frontier closed-source model is reduced to 2.5 CGRE points.

By problem category: Largest CGRE gains occur on array and DP problems (44.2% CGRE), where efficient algorithmic patterns (prefix sums, memoization) are learnable. Smaller gains on graph problems (28.1%), where efficiency requires sophisticated data structures harder to induce from supervision.

Ablations and Interpretation

Reward annealing: Fixing $\alpha = 0.5$ throughout reduces CGRE by 8.2 points, as the efficiency signal disrupts early correctness learning and produces more incorrect programs.

Reference quality: Using a 10th-percentile runtime reference (suboptimal) reduces CGRE by 4.1 points—reference quality is a primary bottleneck and should be set to the best known solution.

KL penalty: Removing D_{KL} causes collapse onto 3–5 high-efficiency code templates, reducing CGRE diversity and failing on problems that do not fit those templates. Performance drops by 6.3 CGRE.

Number of rollouts: 8 rollouts per problem per step is the sweet spot; 4 rollouts degrades CGRE by 3.1 points due to lower variance in the reward signal.

Reference

Huihao Jing, Haozhe Cui, Wenbin Hu, Shaojin Chen, Haochen Shi, Changxuan Fan, Yuxuan Liu, Hanyu Yang, Sirui Zhang, Ziyi Chen, Haoran Li, Yangqiu Song. **RLPF: Reinforcement Learning from Performance Feedback for Code Generation.** arXiv:2607.27271, July 29, 2026. <https://arxiv.org/abs/2607.27271>